

BootLoops: an LLM-driven toolkit for exact quantitative science

Matthew D. Schwartz
Department of Physics, Harvard University

October 1, 2026

Abstract

Much of quantitative science involves difficult mathematical computations: a Feynman integral, evidence for an evolutionary tree, or a regulatory statistic. These are often estimated by Monte Carlo or heuristic search, or computed in floating point without tracking rounding error. Exact methods often exist, but in a field far from the problem. The reduction methods of collider physics also work on a Bayesian evidence, and a regulatory threshold can be settled with interval arithmetic from numerical analysis, but few people know both fields. A large language model knows both, and every other field, and with working programs in hand it can carry a method from one to another. BootLoops is a pool of such programs: integral reduction, differential equations and high-precision evaluation from particle physics, integer-relation fitting from experimental mathematics, exact enumeration, and ball arithmetic, in one toolkit that any agentic large language model can operate and extend. These tools can compute multi-loop Feynman integrals by bootstrap methods rather than direct integration, across the function classes that arise, including polylogarithms, elliptic functions, and periods of K3 surfaces and Calabi–Yau manifolds. They can also evaluate Bayesian evidence integrals as exact rational numbers, enumerate finite configuration spaces with proof of completeness, show when a sought closed form cannot exist, and redo floating-point calculations with guaranteed digits. In practice this brings the tools of mathematics, computer science and physics to problems in genomics, statistics, ecology and other fields. The toolkit and its documentation are at <https://bootloops.ai>.

1 Introduction

For most of the history of scientific computing, a human being would write a program, knowing exactly what it does, and then a computer would carry it out. With machine learning, this paradigm changed: a human would specify the architecture, a computer would fit a neural network, and the human would interpret the results. For example, AlphaFold predicts the structure of a protein from its sequence [1], GraphCast produces medium-range weather forecasts [2], and networks trained on known crystals predict new stable materials [3]. These neural networks can stand in for a calculation that is too expensive to run or that nobody knows how to write down otherwise. Now the paradigm has changed again: a large language model running as a semi-autonomous artificially intelligent agent can design and write the program, run it, and review, rewrite and interpret the answer itself. The potential of such agentic systems seems unbounded, yet the mechanism by which humans can be maximally productive using agentic AI remains uncertain. There is, however, one clear path by which the utility of agentic AI can be enhanced: allowing the models to bring together techniques from distant fields. In particular, methods developed for computer science, mathematics and physics can be directly relevant for problems in population genetics or ecology, yet few people know both sides well enough to connect them. This paper introduces BootLoops, a harness for

large language models to use that brings together various packages for scientific computing in one place.

One of the great strengths of large language models is the breadth of their knowledge. In science, it may be possible to compute a quantity of interest exactly, or with a proven bound on its error, by a method that already exists in a field far removed from the problem, which no human has noticed but an AI can find. For example, the sophisticated reduction methods that physicists and mathematicians developed for Feynman integrals apply directly to Bayesian evidence integrals needed by statisticians and biologists. Or an interest group may want to know if some published data clears a regulatory threshold after being processed through a mandated sequence of algorithmic steps, but the rounding error grows uncontrollably. With interval arithmetic, a tool from numerical analysis that carries a proven error bound through every step, a rigorous reading may be possible. Making these kinds of connections requires understanding the mathematics or computer science from one field and the questions of another, connections that may be hard for humans to make but easy for large language models.

Even if language models are capable of making the connections, they still need to implement and port the methods from one field to another. This takes work. But the work does not have to be repeated over and over again by every scientist interested in the connection. The imported and novel code can be centralized and made accessible. BootLoops serves this purpose: it brings programs for quantitative science together from distant fields so that a language model can run them. These programs reduce and evaluate integrals, solve their differential equations, fit exact constants to high-precision numbers by integer-relation methods, enumerate finite sets exhaustively, and compute in exact and in ball (interval) arithmetic. Some of them are public codes, which we adopted as released or, where they were written as Mathematica packages, translated to run on our own open numerical libraries (Section 2.2). More of the work went into improving existing codes, which the model, Claude (Anthropic), extended and in places rebuilt when their numerical methods failed at the precision required. Most of the programs, however, are new, and many of them implement methods described in papers but never released as code, for example in the theory of modular forms and in algebraic statistics. The others the model wrote from scratch, where no suitable method existed. For the Feynman-integral computations alone, roughly one hundred new programs were written.

The initial version of BootLoops has tools for many computations in four broad categories. First, it has tools to evaluate integrals, such as for Feynman diagrams of scattering amplitudes, in closed form by bootstrap methods, without direct integration. Past one loop the answers are in general no longer polylogarithms, and the functions that appear are governed by elliptic curves, K3 surfaces and Calabi–Yau manifolds [4]. In the S-matrix bootstrap one writes down the most general function in an allowed space and lets the known singularities and limits fix the free coefficients [5]. BootLoops combines the Landau bootstrap, which uses the singularity locus of a single integral [6], with analytic regression, which fits a formula to high-precision values by integer-relation methods [7], for a powerful semi-numerical bootstrap apparatus. An advantage of the semi-numerical approach is that each closed form can be checked against an independent numerical evaluation of its integral to hundreds of digits if needed. The same machinery developed for scattering amplitudes also works for period integrals in cosmology, gravitation and string theory. Second, BootLoops can compute Bayesian evidence integrals for count data under a polynomial model. These integrals are the same mathematical objects as Feynman integrals in their Lee–Pomeransky representation, and BootLoops can evaluate them as exact rational numbers for many statistical models. Third, BootLoops can enumerate finite configuration spaces with a rigorous proof that no case was missed. Where a closed form does not exist, BootLoops can prove it. Finally, BootLoops can redo general floating-point calculations in ball arithmetic, so that error propagation is under control and exact

bounds can be established.

BootLoops has many applications. Phylogeneticists compare candidate trees by their Bayesian evidence [8], as do statisticians for mixture models [9]. Related likelihoods arise in population genetics, in ecology and in models of transcription in single cells [10]. String theorists counting flux vacua [11] face a finite set of cases that can be enumerated completely, as do the designers of an adaptive clinical trial [12]. Exact and ball arithmetic can also be used when numbers follow from a stated rule, as in Medicare’s star ratings [13] and the U.S. ozone standard [14].

Building a harness to make agentic AI more effective is not a new idea. For example, FunSearch pairs a language model with a program that scores the model’s proposals, leading to new constructions in extremal combinatorics [15]. The AI Scientist runs a language model as an agent through a machine-learning project, from the idea to the written paper [16]. More recently, Robin, a system of several agents built at FutureHouse, ran the cycle of literature search, hypothesis, experiment design and data analysis for a problem in biology and proposed a drug candidate for dry macular degeneration that laboratory experiments then supported [17]. These systems, however, are aimed at autonomous science. BootLoops is aimed at a language model working under human supervision. It is a collection of tools that people would use if they knew of them, and the language model makes them easy to use even for someone who does not fully understand how they work. In this it resembles Claude Science, a harness that Anthropic built to make agentic AI easier for scientists to use [18]. Importantly, though, BootLoops can be used with any underlying language model, or inside a commercial harness such as Claude Science or Codex (OpenAI) [19], and it is open source and can be extended by the community of scientists who use it.

No single ingredient of the methods in BootLoops is new. The reduction algorithms and high-precision numerics come from collider physics, the machinery of periods from mirror symmetry, the modular-form and integer-relation methods from number theory and experimental mathematics, and the evidence integrals from statistics. Interval arithmetic and exhaustive search with proof are standard tools of numerical analysis and computer science. Our contribution is to pool these methods as working programs in a single toolkit, BootLoops, which a language model can operate and extend because it knows all of these fields. Section 2 describes the toolkit and its growth, Sections 3–6 the four kinds of computation in turn, and Section 7 concludes.

2 The toolkit

A computation with BootLoops involves a large language model and the toolkit of deterministic programs that the model operates and extends. The programs implement methods of particle physics, experimental mathematics, computer algebra and numerical analysis (Table 1), and we use them for the four kinds of computation listed in Table 2. This section describes the toolkit and contains no new results.

2.1 The model and the toolkit

The computations reported here were run by a large language model, Claude (Anthropic), with a human setting the targets and the standards. The model runs as an agent, that is, in a loop in which it issues commands that execute programs and then reads their output. Given an object to compute, the model plans the calculation and runs the programs that each step requires. On a new integral family, for example, it classifies the geometry (sphere, elliptic curve, K3 or Calabi–Yau), proposes the space of functions in which the answer should lie, and runs the reductions, transports and fits that fix the answer there. When the existing tools lack an operation that a problem requires, the model writes a new routine, tests it on cases with known answers and adds it to the

toolkit. The programs compute every number and symbol. At no step is the model asked for a digit, and no result rests on the model’s judgment.

The toolkit persists across problems. It holds the programs that reduce an integral family to a finite basis, solve its differential equations, evaluate it numerically, fit function spaces to the values, integrate with proven error bounds and enumerate configurations exhaustively. The programs share formats, and the output of one becomes the input of the next. None of this changes the model itself: the weights never move. The improvement from one problem to the next accumulates in the toolkit and its documentation. A harness, as the term is used for language models, is the layer of software through which a model is operated. BootLoops is a living harness in this sense, since people and the model extend it with every problem while it stays separate from the model.

2.2 The toolkit by class and its growth

Table 1 lists the principal programs of the toolkit in nine classes by function. The toolkit’s external part consists of public programs. Some, such as the reduction program Kira [20] and the Landau-singularity program PLD [21], are used as released (Kira through a performance-patched fork whose reduction output is unchanged). Others, such as the singular-locus program SOFIA [22] and the reduction program Blade [23], were published as Wolfram Language packages. Such a package is its authors’ own code: it implements the physics and calls Mathematica’s built-in routines for generic tasks such as arbitrary-precision arithmetic, linear algebra, the series solution of differential equations, polynomial algebra and numerical integration. To port such a package we wrote our own libraries for those generic tasks, in C++ and Julia, from the standard published algorithms, and translated the authors’ package code to call them. A port therefore consists of the package authors’ algorithms running on our open libraries; SOFIA runs this way in Julia and Blade on a compiled toolchain. For auxiliary-mass flow [24] a public C++ reimplementation, AMFlow.cpp [25], already existed, and we adopted it as a fork that we extended. No required step then needs a Mathematica license, and a language model can read, run and modify every layer of the program. No attempt was ever made to reverse-engineer any routine in a commercial package; code for ports was written from scratch using public algorithms.

Where we changed a public open-source program’s numerical methods or added routines, we keep the changed version as a fork (a full copy of the original project carrying our patches). The original code, license and attribution then stay intact, and the patches can be offered back. The internal part is the larger one, and we wrote it in the course of the work. For the Feynman-integral computations alone this amounted to roughly one hundred purpose-built tools where no public equivalent existed.

New tools were added to the toolkit in four ways. A public program was ported in the sense defined above, as with SOFIA and Blade. A program reached a limit in the middle of a problem and was upgraded (the upgrade kept as a fork). For example, we rebuilt the series solver of the AMFlow.cpp fork to keep nearly coincident solution families separated at very high working precision. The model turned methods that existed only in papers into code: Eichler evaluates the Eichler integrals (iterated integrals of modular forms [39]) that appear in elliptic Feynman integrals, and Mixalot computes the evidence of a mixture model exactly by methods from algebraic statistics [40].

Finally, a tool was built from scratch where nothing existed. Public programs find where an integral is singular [21, 22], but none that we found turned that singular set into the working ansatz of the bootstrap (the alphabet, the analytic conditions on it, and the count of coefficients left to fit). We wrote the pair Landau Alphabet and Ansatzer for this step. The interval routine ERAS (Section 6.2) was written for the same reason during a statistical application, when plain interval

Table 1: The toolkit by function: in each class the mature public programs were adopted as released or ported, and the remaining programs were written in the course of the work where nothing public covered the need. Under each class name we give the sections where the class is used and, for a program reused in another kind of computation, the section where it is reused. Names are the programs’ public names; every program has a documentation page in the online registry at <https://bootloops.ai> (Section 7). Programs of the last class check the output of the other eight.

Class (used in)	What it computes	Adopted or ported	Written for BootLoops
Reduction engines (Sec. 3)	reduction of the thousands, sometimes millions, of integrals in a family to a finite basis by integration-by-parts identities; the memory a reduction will need, predicted before it runs	Kira [20] with FireFly [26] and Fermat (a performance-patched fork, reduction output unchanged); Blade [23], ported to run without Mathematica	Seedling [27] (staged seeding, cost prediction); Winnow (each identity with its own proof; one sector’s differential equation from a family too large to reduce whole)
Singularities and geometry (Sec. 3)	the complete set of points where an integral can be singular, the symbol alphabet it implies, and the geometry of the maximal cut (sphere, elliptic curve, K3, Calabi–Yau) that fixes the class of functions	PLD [21], as released; SOFIA [22], ported to Julia	Leviathan (independent second computation of the singular set); Landau Alphabet (singular set to alphabet); GeoTriage (geometry from the Picard–Fuchs operator); Maxcut
Differential equations and transport (Sec. 3; Annihilator reused in Secs. 4, 5)	canonical form of a differential-equation system; the connection reconstructed from numerical samples; a value carried from one kinematic point to any other along the equation; the annihilating differential or recurrence operator of a series, with minimal order proven	none	Counterweight; Wayfinder; Vopclose; Annihilator
Evaluators (Sec. 3)	master integrals to hundreds of digits; polylogarithms and elliptic, K3 and Calabi–Yau periods to any requested precision; quadrature with a proven error bound	AMFlow.cpp [25], the public C++ reimplementation of auxiliary-mass flow [24], forked and extended; pySecDec [28] and FIESTA [29], as released, kept because they share no code with the rest	Nestor (quadrature); GPLeval (polylogarithms); Eichler (periods)
Closure and naming (Sec. 3)	the exact combination of declared constants that a high-precision value equals, under a stated bound on the integer coefficients, or the statement that none exists below the bound	PSLQ [30] and LLL [31]	Ansatzter (the ansatz and its residual count); Lockpick (lattice reduction where PSLQ stalls); RingMaker (the admissible ring of constants); Frobenius (boundary constants by arithmetic modulo primes)
Ball arithmetic (Secs. 4, 6)	every real quantity as a midpoint with a proven radius; quadrature, transport and root isolation with proven bounds; usable bounds when one parameter enters every factor of a large product	Arb [32] and mpmath [33]	Baller [34]; ERAS
Evidence and statistics (Sec. 4)	the Bayesian evidence of a tree or a mixture model as an exact number or a proven enclosure; population-genetic likelihoods with proven error bounds	none	JaCK & Jill [35]; Mixalot [9]; Popcorn
Enumeration and audits (Secs. 5, 6)	a complete enumeration with a proof that nothing was missed; the exact replay of published arithmetic from the publication’s own inputs	none	Terrier [11] (flux lattices, exact lattice arithmetic); Recount (replay of published numbers; distributed with the audits)
Integrity (all sections)	the precision and provenance of every reference value; reduction output confirmed by checks with disjoint code; an integral family confirmed against its graph before any reduction runs	none	Gatekeeper; Trust; the family preflight of Seedling

Table 2: The four kinds of computation of Sections 3–6, all carried out with the programs of Table 1: the object computed exactly in each, the usual practice in the fields concerned, and the areas of application. Results are stated in this paper for the first row and, in Section 5.2, for the Watson integral of the third; otherwise Sections 4–6 describe the computation and its relevance to the areas named without reporting results.

Kind of computation	Exact object	Usual practice in the fields	Application areas
Function-space bootstrap of Feynman-class integrals (Sec. 3)	a closed form $F(s)$ for an integral of the class of Eq. (2): named functions on a stated alphabet, with every constant classical or given by a convergent integral, and a standalone evaluator	numerical evaluation point by point [24]; closed forms derived case by case	collider amplitudes; energy correlators; gravitational-wave scattering; cosmological correlators; large-scale structure
Exact evidence and the holonomic class (Sec. 4)	the evidence Z of Eq. (8) as an exact rational number or an element of a stated ring; more generally a period or holonomic function together with its annihilating operator	Monte Carlo estimates of the evidence [36, 37, 38], whose error bars are themselves estimates	phylogenetics; mixture models and authorship; forest dynamics; single-cell transcription kinetics; population genetics
Exhaustive enumeration and proof (Sec. 5)	the complete list of configurations a theory allows, or a theorem that no object of a stated form exists (an operator of proven minimal order)	sampled or heuristic search; conjectured closed forms tested numerically	string flux vacua; Voynich readings; the Watson integral; clinical-trial operating characteristics; clustering optima of rating rules
Ball arithmetic and audits (Sec. 6)	a real number as a ball $m \pm r$ of Eq. (13) whose radius is a proven bound; for a published number, its recomputation from the publication’s own inputs	floating-point arithmetic whose rounding error is not tracked; reruns at higher precision by hand	Medicare star ratings; the ozone standard’s truncation arithmetic; population-genetics reference software; periods of string compactifications

arithmetic returned unusably wide bounds on a likelihood.

A tool built for one problem has usually been reused in later ones, either unchanged or extended. Eichler was written to evaluate one function, was then extended to the periods of a family of elliptic curves, and later to K3 and Calabi–Yau periods. Annihilator, written to find and certify the differential and recurrence operators of Feynman-integral series, computes the recurrences of evidence integrals unchanged. The routines and documentation written during one problem remain in the toolkit for the next. Because the toolkit is separate from the model, any agentic large language model, including a later one, should be able to operate and extend it as it stands.

2.3 Documentation and instructions

Every program in the toolkit has a documentation page, and the toolkit is distributed with plain instruction files that any model can read. These state what each tool does, when to reach for it, what its output means and how its answer is tested. The intended use is that a large language model will read these files first. If the tools exist that it needs for its project, great. If not, it locates where an existing tool applies and extends that tool instead of writing another from scratch if possible. Working protocols for good scientific practice and agentic AI use are distributed with BootLoops as well. For example, one protocol says to try a method on planted answers before real data. Another recommends recording the source and precision of every reference value.

Being able to check and validate results in a semi-autonomous way is critical to productive use of agentic AI. The validation required naturally varies with the kind of computation. One particularly clean example described at length in this paper is the check on closed forms for integrals (Section 3.3). There one may use numerical evaluation to help find the analytic answer, but the result then must be validated to many digits of precision beyond what was used in the computation.

3 Feynman integrals and amplitudes

Loop integrals in quantum field theory can be computed in closed form without direct integration, from high-precision numerical values and constraints that the answer must satisfy. Fixing an answer from general principles alone is an old idea in particle physics. In the 1950s and 60s physicists tried to determine scattering amplitudes from analyticity and unitarity without a Lagrangian, an approach that Geoffrey Chew called the bootstrap [41]. In the perturbative version used here we determine one integral at a time. We describe the class of integrals, the bootstrap and its acceptance test, and the thirty integrals computed so far.

3.1 The problem class

A scalar Feynman integral with L loops and N internal lines is, in momentum space,

$$I(a_1, \dots, a_N) = \int \prod_{\ell=1}^L \frac{d^d k_\ell}{i\pi^{d/2}} \frac{1}{D_1^{a_1} D_2^{a_2} \dots D_N^{a_N}}, \quad D_j = q_j^2 - m_j^2 \quad (1)$$

with one integration per loop momentum k_ℓ . Each internal line contributes a propagator D_j , the factor for a virtual particle of mass m_j and momentum q_j (a fixed combination of loop and external momenta), raised to an integer power a_j . In $d = 4 - 2\varepsilon$ dimensions divergences become poles in ε , and each order in ε is a function of the kinematic invariants s (masses and external momenta).

Feynman parametrization brings every such integral to the form

$$I(s; \varepsilon) = \int_{\Delta} d^n x \prod_i P_i(x; s)^{\nu_i + a_i \varepsilon}, \quad (2)$$

a generalized Euler integral [42]. Here Δ is a semialgebraic domain in the n variables x , the P_i are polynomials, and the exponents $\nu_i + a_i\varepsilon$ are fixed for each problem and linear in ε . For a Feynman graph the P_i are the two Symanzik polynomials $\mathcal{U}$ and $\mathcal{F}$, built from the spanning trees and two-forests of the graph with the invariants s as coefficients. Their exponents follow from the propagator powers a_j of Eq. (1), the number of loops L and the dimension d . In the Lee–Pomeransky representation the single polynomial $\mathcal{G} = \mathcal{U} + \mathcal{F}$ suffices [43].

The possible singularities of $I(s; \varepsilon)$ in s are known in advance. They lie on algebraic varieties, the Landau loci, where the zero sets of the P_i can pinch the integration domain, and are computable from the integrand alone [6, 44]. The class (2) is also holonomic: integration by parts (IBP) yields linear relations among integrals whose exponents differ by integers, and these relations form a closed system. In particular, every family (fixed propagators D_j , all integer powers a_j) is spanned by finitely many *master integrals*, which satisfy a first-order linear system of differential equations in s with rational coefficients [45, 20].

The function space of such an integral is fixed by the algebraic variety that the zero sets of the P_i define. The maximal cut of a master integral (its residue on the poles of all propagators at once) is a period of this variety. Its Picard–Fuchs operator (the differential operator in s annihilating the period) identifies the type of the variety. A rational variety gives multiple polylogarithms [46, 47]; a modular elliptic curve, iterated integrals of modular forms [48, 49]. The K3 surfaces and Calabi–Yau varieties next in dimension give the periods computed in mirror symmetry [50, 51], and the boundary constants of the differential equations are periods too [52].

Closed forms for these integrals are hard to produce and easy to check. A Feynman integral in the class can usually be evaluated to any needed precision by auxiliary-mass flow [24], independently of any closed form. This method gives the propagators an auxiliary mass and transports the integral numerically through its differential equation from large mass to zero.

3.2 The function-space bootstrap

On a single integral the bootstrap (Fig. 1a) is the two-sided squeeze of Fig. 1b, which we run in five stages. First, we compute the complete Landau singular locus from the graph alone [21, 22]. Its components give the alphabet of the answer, a finite list of letters (algebraic functions of s). Second, we classify the maximal cut of each master integral as genus zero (a sphere), elliptic, K3 or Calabi–Yau, which fixes the iterated-integral kernels. With the alphabet and the leading singularity (the algebraic prefactor), the kernels generate a weight-graded ansatz: every allowed function of the right transcendental weight (one for a logarithm, two for a dilogarithm) with an unknown coefficient. The routines for these stages are Landau Alphabet and Ansatzer. Third, exact constraints (branch points only on the Landau loci, first-entry conditions, integrability, the symmetries of the graph) reduce the ansatz to 10^2 – 10^3 unknown coefficients. Indeed, for polylogarithmic master integrals these constraints often fix the answer outright.

Fourth, we constrain the surviving unknowns with high-precision samples of the integral, from auxiliary-mass flow [24] on an IBP reduction [20] or, beyond genus zero, from transport of the periods with rigorous error control. Fifth, we recognize the constants by integer-relation fits. Given real numbers $x_1, \dots, x_n$ known to D digits, the PSLQ and LLL algorithms [30, 31] find integers a_i , not all zero, with

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = 0 \tag{3}$$

to the working precision, or establish that no relation exists with every $|a_i|$ below a stated height H . We accept a relation only if its height is far below the largest that D digits can resolve, it is stable at two working precisions, and a planted control is recovered. Where the acceptance test below cannot be passed, we report what was established instead: a value carried along the differential equation

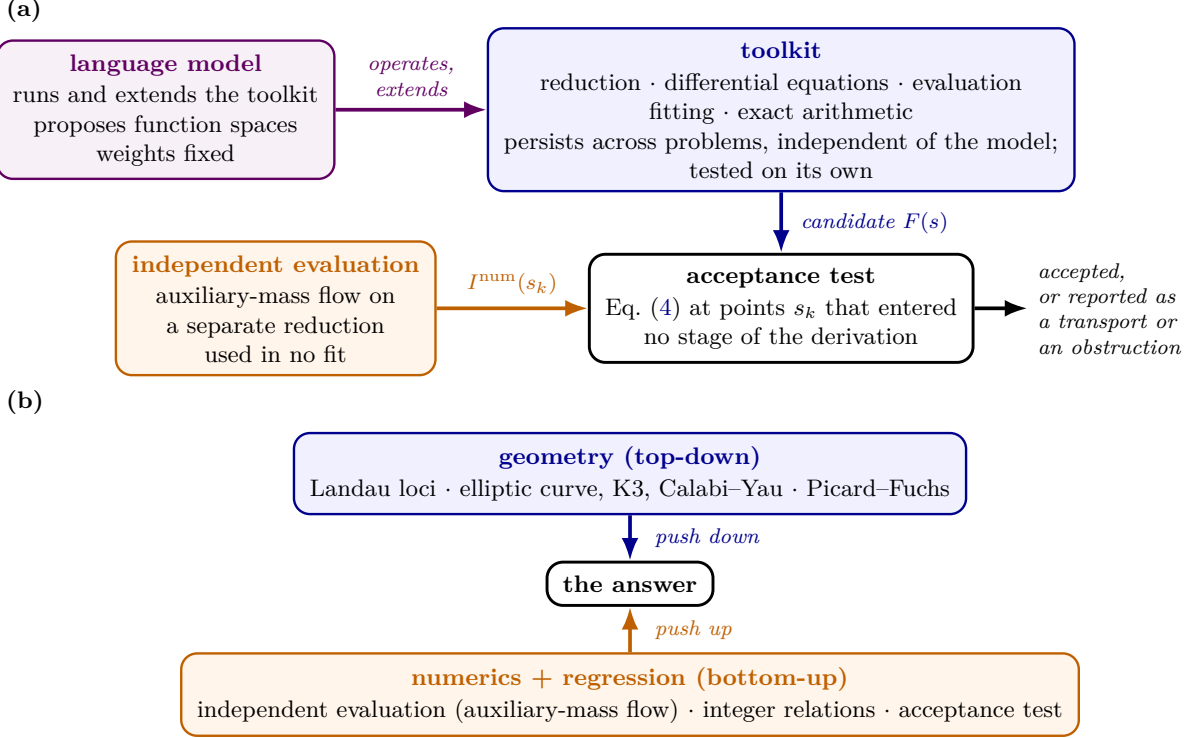


Figure 1: The bootstrap of one Feynman integral with the toolkit. (a) The parts of the computation. The model operates and extends the toolkit, which persists across problems and is separate from the model. The programs produce a candidate closed form $F(s)$, which is compared with an independent numerical evaluation $I^{\text{num}}(s_k)$ (auxiliary-mass flow [24] on a separate reduction [20], used in no fit) at points s_k that entered no stage of the derivation, Eq. (4). (b) The squeeze on one integral. The geometry of the singularities bounds the space of candidate functions from above, and high-precision samples with integer-relation fits fix the remaining coefficients from below.

with a proven error bound, a closed form with some constants left as their defining integrals, or a proof that no function of the proposed class equals the integral.

The five stages form the *constraint leg* of the method. The other leg is the *integration leg*: reduce to master integrals, then solve their differential equations from an exactly known boundary value. That is, we can also compute the integral by solving its differential equations. Sometimes this is the easiest route, and BootLoops has many packages built and ported for this route as well as for the bootstrap. We assign each master integral to a leg before any sampling, chiefly by whether its values can in principle be fit. An elliptic curve has two independent periods, and the maximal cut supplies only the first. Thus if the normalization of a master integral shows it mixes the two periods, a fit with constant coefficients provably cannot match it at all kinematic points, and that master is assigned to the integration leg.

Three further routines serve both legs: rotation of the differential equations to an ε -factorized basis where one exists [47, 53], reconstruction of exact rational functions from finite-field evaluations [54], and representation of an integral with no closed form by its annihilating differential or difference operator, found modulo primes, lifted to the rationals and proved by creative telescoping [55].

3.3 The acceptance test

Let $F(s)$ be a candidate closed form for an integral of the type (2). Let $I^{\text{num}}(s)$ be an evaluation of the same integral obtained independently of the derivation of F , in most cases auxiliary-mass flow [24] on a separate reduction [20]. We accept F when

$$\frac{|F(s_k) - I^{\text{num}}(s_k)|}{|I^{\text{num}}(s_k)|} < 10^{-30} \quad \text{for every held-out point } s_k, \quad (4)$$

where the s_k are kinematic points that entered no stage of the derivation. Agreement of two independently computed real numbers to thirty digits is a coincidence with probability $\sim 10^{-30}$, and each acceptance amounts to a hypothesis test at that significance. Thirty digits is the floor, and many results are tested at sixty or a hundred. Keep in mind that the evidence for an accepted closed form is numerical, in the tradition of high-precision identifications [45, 56]; thus integrals computed this way do not constitute a symbolic proof.

A numerical value of an integral, however precise, does not show which functions its closed form is built from. The **BootLoops standard** for Feynman integrals requires three things of a computed integral: an explicit symbolic form over a declared alphabet, with every constant classical or defined by a written-out convergent integral; a standalone evaluator of that form at any kinematic point and precision from arbitrary-precision arithmetic alone; and Eq. (4), passed at two working precisions. Auxiliary-mass flow supplies the samples of the fourth stage and, from a separate reduction, the independent side of Eq. (4); it also serves on some transport routes, and it never appears inside a closed form itself.

We search for a classical name for every boundary constant, but one need not exist. The period of a generic elliptic curve, one without complex multiplication whose family over the kinematic line is not modular, is not expected to lie in any classical ring. When no relation is found below a stated height, the constant enters the closed form through its defining integral [57].

3.4 Thirty integrals in closed form

With BootLoops we have computed thirty Feynman integrals, from one to four loops, in symbolic closed form [57, 58, 59], ranging over polylogarithmic, elliptic, K3 and Calabi–Yau geometries. All thirty (Table 3) meet the standard of Section 3.3. Fifteen of the thirty were already computed in the literature, among them the four-loop equal-mass banana. For these fifteen the published closed forms were used only at the final comparison, where all fifteen agree; what each computation took from the literature, if anything, is stated in its entry in Ref. [57]. The other fifteen are new: we have found no prior computation of them in the literature [57]. Three of the known and twelve of the new integrals are drawn in Fig. 2. Twenty-four of the thirty were fixed by geometry, analytic constraints and integer-relation fits, and six went through the integration leg. Table 3 lists each integral’s agreement with held-out values (one qualification appears in its caption), and Ref. [57] contains one derivation per graph.

Table 3: The thirty Feynman integrals computed in closed form, by the geometry of the maximal cut, numbered as in Ref. [57]. Fifteen were known and fifteen are new, with no prior computation that we could find (three and twelve of these appear in Fig. 2). The order- ϵ^1 sunrise and ice-cream cone count as rows. Digits: agreement with comparison values that entered no fit (the floor is thirty; most elliptic rows reach 100 or more). One qualification: for Nos. 2 and 4 the digits quoted are agreement with the analytic results of the literature, the auxiliary-mass-flow comparisons of those two rows reaching ≥ 89 and 72 digits. ^aRational coefficients of the published form reproduced exactly. ^bPrecision to which the closed-form identification is verified in ball arithmetic [59]; the independent check of these two rows, against the IBP operator (Ref. [57]), reaches 47 and 45 digits.

Integral	Description	Digits
<i>Polylogarithmic (maximal cut of genus zero)</i>		
1 outer-mass double box	planar two-loop double box, one common mass on the six perimeter lines, massless legs	≥ 86
2 massless double box	planar two-loop double box, all lines massless	exact ^a
4 three-loop ladder	planar massless ladder at $L = 3$	≥ 461
5 one-loop pentagon	massless, five on-shell legs; 15 letters already at weight two	68
6 Sudakov vertex	two-loop massless ladder vertex, one off-shell leg, a single scale	≥ 78
7 central-mass double box	two loops, four massless legs, one massive central rung; three scales, seven letters	109–112
23 massless-rung kite, four masses	two-loop kite with squared masses $(1, 4, 9, 2, 0)$; seven letters over four square roots	81–110
24 non-planar hexa-box	two-loop five-point, off-shell legs $p_1^2 = p_3^2 = m^2$; 98 master integrals	66–68
<i>Elliptic (maximal cut an elliptic curve)</i>		
8 equal-mass sunrise	two loops, three equal masses, order ϵ^0 ; the $\Gamma_1(6)$ curve	140–159
9 sunrise $(1, 1, 2)$	squared masses $(1, 1, 2)$, order ϵ^0	159
10 sunrise $(1, 2, 3)$	squared masses $(1, 2, 3)$, all three marked points distinct	159
11 sunrise $(1, 1, 4)$	squared masses $(1, 1, 4)$	159–289
12 sunrise at ϵ^1	the order- ϵ^1 term for the three unequal-mass assignments	288–308
13 equal-mass kite	the equal-mass sunrise with a massless rung across it; the $\Gamma_1(6)$ curve	130–160
14 unequal-mass kite	non-modular sunrise sub-curve; solved by transport from $s = 0$	≥ 125
15 equal-mass ice-cream cone	two-loop three-point graph with four massive lines, order ϵ^0	134
16 generic-mass ice-cream cone	squared masses $(1, 2, 3, 5)$; non-modular family of curves	118
17 ice-cream cone at ϵ^1	the weight-four order, on both mass slices	134
18 $gg \rightarrow H$ elliptic masters	two-loop mixed QCD–electroweak; the two-mass sunrise curve (m_t, m_t, M)	48
20 $gg \rightarrow Z\gamma$ at exact m_t	the pair of elliptic masters; boundary an Eichler integral on $\Gamma_0(4)$	109–112
27 light-by-light (sunrise)	three loops: the one-loop massive box with the equal-mass sunrise inserted on one rung	≥ 45
28 light-by-light (kite)	the massive box with the equal-mass kite inserted on one rung; dispersive	59
29 light-by-light (QED parent)	the ten-propagator parent of the kite insertion, with two electron stubs	50, 68
33 light-by-light (N_f^2)	the box with a vacuum-polarization insertion; dispersive, exact in d	≥ 37
35 $q\bar{q} \rightarrow W^+W^-$ non-planar	two loops with exact top-quark mass, on a kinematic line; 76 masters, 23 new	37–57
<i>K3 surface</i>		
19 three-loop banana	four equal masses, order ϵ^0 ; boundary constant $7\zeta_3$	≥ 110
31 threshold banana (K3)	three loops, squared masses $(1, 1, 1, 9)$; connection coefficient $c_{3/2} = -\sqrt{3}/(36\pi)$	195 ^b
<i>Calabi–Yau threefold</i>		
34 four-loop banana	five equal masses, order ϵ^0 ; operator AESZ 34	116
32 threshold banana (CY)	four loops, squared masses $(1, 1, 1, 1, 16)$; coefficients in the $\Gamma(\frac{1}{4})$ ring	197 ^b
<i>Polylogarithmic top sector over a K3 sub-sector</i>		
21 crossed light-by-light box	three loops: massive fermion loop, two crossed massless photon diagonals, four on-shell photons	33–72

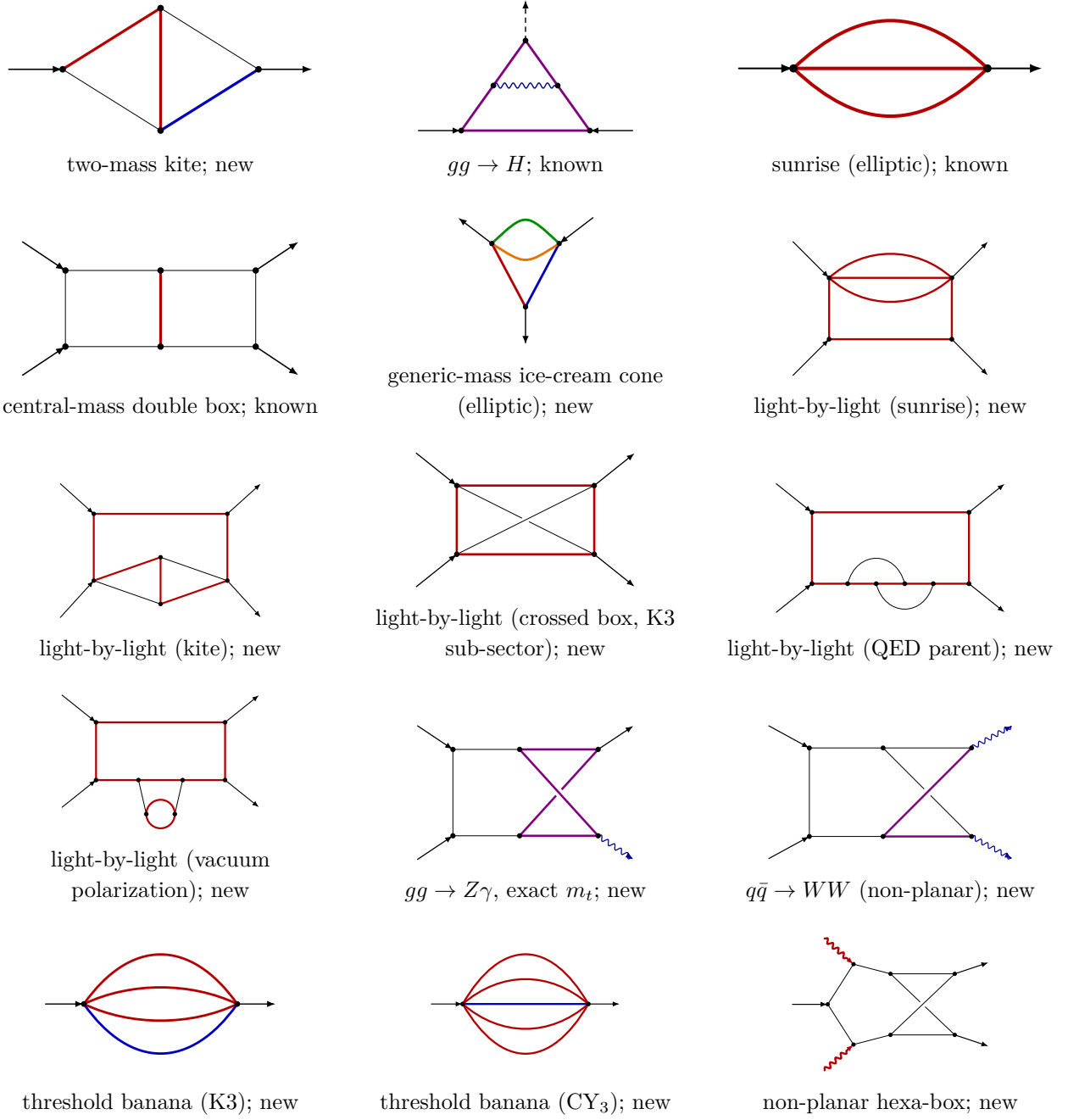


Figure 2: Twelve of the fifteen integrals drawn here, marked “new”, have no prior computation we could find; the three marked “known” ($gg \rightarrow H$, the equal-mass sunrise and the central-mass double box) are literature integrals, with the published results used only at the final comparison. Tile labels abbreviate the names in Table 3 (the two-mass kite is its unequal-mass kite, the vacuum-polarization insertion its N_f^2 row); elliptic, K3 and CY₃ (Calabi–Yau threefold) name the geometry of the maximal cut where shown. Doubled or colored lines are massive, one color per distinct mass; thin black lines are massless, and a colored wavy external leg is off shell; in the $gg \rightarrow H$ tile the dashed leg is the Higgs boson and the wavy internal line the electroweak boson of mass M .

4 Evidence integrals

In this section, we explain how the Bayesian evidence of a polynomial model for count data is an integral of the same kind as for a Feynman diagram, so that it can be evaluated with the same tools (Section 3). For the applications below (Table 2) the findings are in the papers cited.

4.1 Bayesian evidence integrals

Bayesian model selection ranks competing hypotheses by their evidence, or marginal likelihood: the probability of the data under a hypothesis, integrated over its parameters against their prior [60]. In general a model M with parameters θ assigns the data D a likelihood $P(D|\theta, M)$, and a prior density $\pi(\theta|M)$ weights the parameters. Bayes’ theorem gives their posterior distribution,

$$P(\theta|D, M) = \frac{P(D|\theta, M) \pi(\theta|M)}{Z_M} \quad (5)$$

with normalization

$$Z_M = \int P(D|\theta, M) \pi(\theta|M) d\theta, \quad (6)$$

called the *evidence* or the *marginal likelihood*. The evidence allows us to compare models: the posterior odds of M_1 over M_2 are the prior odds times the Bayes factor Z_{M_1}/Z_{M_2} . As a toy example, toss a coin n times and observe k heads. Under the hypothesis of a fair coin there is no parameter and $Z_{\text{fair}} = 2^{-n}$. Under the hypothesis of an unknown bias x , the probability of heads, with a uniform prior on $[0, 1]$,

$$Z_{\text{bias}} = \int_0^1 x^k (1-x)^{n-k} dx = \frac{k! (n-k)!}{(n+1)!}. \quad (7)$$

Note that the unknown bias is integrated over: any bias is allowed with a flat prior on x . If the data come out as $k = 8$ heads after $n = 10$ flips, this gives $Z_{\text{fair}} = 1/1024$ and $Z_{\text{bias}} = 1/495$, a Bayes factor $Z_{\text{bias}}/Z_{\text{fair}} \approx 2.1$ in favor of a biased coin. In words, the observed eight heads in ten flips is 2.1 times as probable under the biased-coin hypothesis as under the fair-coin hypothesis. More technically, even prior odds become posterior odds of around 2 : 1, so that the probability is $2.1/(1+2.1) \approx 2/3$ that the coin is biased. A common convention [60] is that a Bayes factor below about three is weak evidence and “not worth more than a bare mention”.

The integrand of Eq. (7) is a product of the outcome probabilities, x for heads and $1-x$ for tails, each raised to the number of times that outcome occurred. The same structure appears whenever the data are counts and the probability of each outcome is a polynomial in the parameters. With outcome k observed u_k times, outcome probabilities $P_k(x)$ in parameters $x = (x_1, \dots, x_d)$ scaled to the unit cube, and a prior proportional to $\prod_e x_e^{\alpha_e-1}$ (uniform for $\alpha_e = 1$), the evidence is

$$Z = \int_0^1 \cdots \int_0^1 \prod_k P_k(x)^{u_k} \prod_e x_e^{\alpha_e-1} dx, \quad (8)$$

with the polynomials P_k raised to the data counts u_k and the exponents α_e set by the prior. In most models of interest it has no elementary closed form. Equation (8) is a generalized Euler integral of the class of Eq. (2) [42], and the counts u_k occupy the place of the propagator powers in the Lee–Pomeransky representation of a Feynman integral [43]. Algebraic statisticians have written statistical models in this polynomial form [61] and computed such integrals exactly for small models [40]. We apply the reduction, recurrence and finite-field routines built for Feynman integrals to Eq. (8) unchanged.

4.2 Phylogenetics

Phylogenetics offers a concrete example. The problem is to infer, from the DNA of several living species, the tree by which they descend from a common ancestor: its branching order (the topology) and the lengths of its branches. The data are aligned sequences, one row per species and one column per site, so that each column records which of the four bases every species carries at one position (Fig. 3b). A substitution model, a Markov chain on the four bases running along the branches, gives the probability of each column as a function of the branch lengths, and the likelihood of the alignment is the product of these probabilities over the columns. Each topology is then a hypothesis M in the sense of Eq. (5) with the branch lengths as its parameters θ , and rival topologies are compared by their evidences. For thirty years phylogeneticists have estimated these evidences by Monte Carlo (thermodynamic integration, stepping-stone sampling, nested sampling), with error bars that are themselves estimates [36, 37, 38]; packages such as MrBayes [62] work this way.

We make the correspondence between evidence integrals and Feynman integrals explicit for the smallest phylogenetic problem. Its hypotheses are the three tree topologies (branching orders) that could relate four species (a quartet), and its parameters are the five branch lengths of the unrooted tree τ (Fig. 3a). The alignment has N sites. We take the Jukes–Cantor model, the simplest base-symmetric substitution model: each site evolves independently along τ by a continuous-time Markov chain on the four bases with all substitutions equally likely. Along an edge of length t (expected substitutions per site) a base survives or changes with probabilities

$$P_{\text{same}} = \frac{1 + 3x}{4}, \quad P_{\text{diff}} = \frac{1 - x}{4}, \quad x = e^{-4t/3}. \quad (9)$$

Because the Jukes–Cantor model treats the bases alike, the patterns of bases that a site can show across the four species fall into classes p , the patterns within a class having equal probability, with N_p sites in class p (Fig. 3b). The probability f_p of a pattern in class p is a sum, over the states of the two internal nodes, of products of five edge factors, each linear in one x_e by Eq. (9). Thus f_p is a polynomial with rational coefficients, multilinear in $x_1, \dots, x_5$, and the likelihood of the alignment is $\prod_p f_p^{N_p}$. We put an independent exponential prior of rate 4/3 on each branch length. Its density cancels the Jacobian of $t \mapsto x$ exactly, so that each x_e is uniformly distributed on $[0, 1]$ and the evidence for the topology is

$$Z_\tau = \int_{[0,1]^d} \prod_p f_p(x)^{N_p} d^d x, \quad (10)$$

with $d = 5$ for a quartet and $d = 7$ for a quintet. Hence Eq. (10) is Eq. (8) with $P_k = f_p$, $u_k = N_p$ and every $\alpha_e = 1$. Each of the three topologies has its own pattern polynomials, and the fifteen topologies of a quintet are scored likewise.

For four- and five-taxon trees under the standard base-symmetric substitution models with exponential branch-length priors, the evidence is an exact rational number, a ratio of explicit integers [8, 35]. It is rational because each monomial $c_m x^m$ of $\prod_p f_p^{N_p}$ integrates over the unit cube to $c_m \prod_e 1/(m_e + 1)$; by multilinearity every exponent m_e is at most $N = \sum_p N_p$, which bounds the denominator of Z_τ in terms of the counts alone. We evaluate the monomial sum modulo machine-word primes and reconstruct the rational by Chinese remaindering [8]; two reconstructions from disjoint sets of primes must then agree. It is also compared with an independent arbitrary-precision quadrature of Eq. (10).

As with a family of Feynman integrals, finitely many master integrals span the family of integrals Z_τ , and members whose counts differ by integers are connected by linear contiguity recurrences with rational coefficients, the counterpart of the integration-by-parts identities. The one-parameter

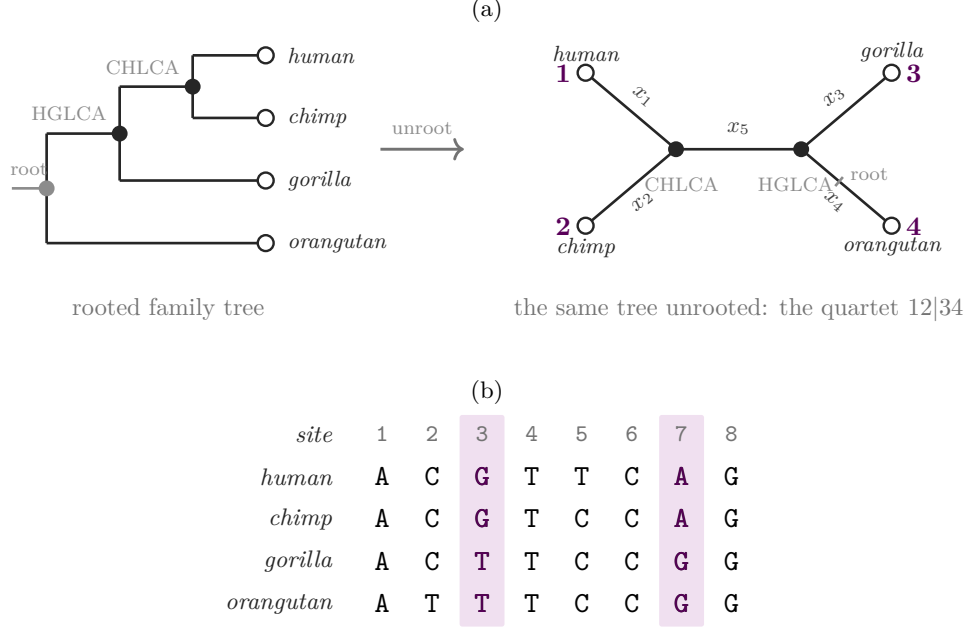


Figure 3: The evidence of a four-taxon tree depends on the DNA alignment only through the counts N_p of site-pattern classes, which occupy in Eq. (10) the place of the propagator powers in a Feynman integral. (a) The rooted family tree of human, chimpanzee, gorilla and orangutan (internal nodes: the chimpanzee–human and human–gorilla last common ancestors, CHLCA and HGLCA) and the same tree unrooted, the quartet topology 12|34 with pendant edge variables $x_1, \dots, x_4$ and internal x_5 , where $x_e = e^{-4t_e/3}$; unrooting leaves the root as a tick mark on the orangutan branch. (b) A short alignment, one column per site; the shaded columns show the pattern class supporting 12|34 against the rivals 13|24 and 14|23.

case is classical, and the coin of Eq. (7) is its simplest instance. For k successes in n Bernoulli trials under a Beta(a, b) prior on the success rate, the evidence $Z(a, b; k, n)$ obeys

$$\frac{Z(a, b; k+1, n+1)}{Z(a, b; k, n)} = \frac{a+k}{a+b+n}, \quad (11)$$

which is the posterior predictive probability of the next success (Laplace’s rule of succession). Read as a computation, Eq. (11) and its counterpart for a failure, with $b+n-k$ in the numerator, give the evidence of any dataset from $Z(\text{no data}) = 1$ in n exact rational multiplications. For the quartet the recurrence has higher order and matrix coefficients. We obtain it with Annihilator (Table 1), the routine written to find the Picard–Fuchs operators of banana-graph periods, used unchanged with shifts of pattern counts replacing shifts of propagator powers [8].

Two topologies can have exactly equal evidence. If a relabeling of the taxa exchanges two rival topologies while preserving the pattern counts, their evidences are equal by symmetry. The exact computation then returns identical rational numbers, whereas a floating-point evaluator gives two nearby numbers and cannot report the tie, which no amount of sampling could establish.

Outside the base-symmetric models with exponential priors (for richer substitution models, or with rate variation across sites) we compute the evidence integral as an enclosure proven to contain it, in the ball arithmetic of Section 6. A comparison of two hypotheses then becomes an inequality between intervals, which is conclusive only when the lower bound on one evidence exceeds the upper bound on the other and is otherwise reported as inconclusive. For the exact computation and the

enclosure alike, the practical limit is cost, which rises steeply with the number of species and of rate classes. The package JaCK & Jill [35, 8] returns these evidences for four species, exactly under the Jukes–Cantor and Kimura models and as proven enclosures otherwise.

4.3 Evidence integrals in other fields

Integrals of the form of Eq. (8) and the recurrences that connect them arise in other fields where a probabilistic model of counts is compared with data. In the statistics of mixtures one asks whether data that look mixed come from one population or several. For a discrete variable the marginal likelihoods of models with different numbers of components, treated as intractable for decades, are again integrals of polynomials raised to the data counts. The BootLoops tool Mixalot [9] evaluates them as finite sums by the method of Ref. [40]. Shared authorship is also a mixture problem. Counts of function words (articles, prepositions and the like) have mostly been used to assign a disputed text to one author. Treating a text as a mixture makes each proposed division among authors a hypothesis with its own evidence in closed form [63].

Evidence integrals arise in several other fields as well. In neutral ecology the likelihood is the probability of the observed species abundances on a forest plot under Hubbell’s model [64, 65]. It obeys an exact recurrence in the abundance counts, although it is usually approximated by Monte Carlo simulation. In single-cell biology the rates and sizes of transcriptional bursts (brief episodes of RNA synthesis separated by long silent intervals) are fitted to RNA counts per cell [66]. The likelihood is a product of hundreds of factors, and standard floating-point tools give no bound on the accumulated rounding error of the product [10]. In population genetics the site-frequency spectrum of a selected allele (how many variants appear on one, two or more sampled chromosomes) underlies the inference of fitness effects. The spectrum predicted by the standard theory (the Sawyer–Hartl Poisson random field) could not be evaluated accurately at strong selection and large sample sizes, where floating-point arithmetic fails [67]. We compute the bursting likelihood in ball arithmetic and the site-frequency spectrum in exact rational arithmetic.

5 Exhaustive enumeration and proof

Some quantitative questions are settled only by examining every case, and others only by a proof that no case of the sought kind exists. Exhaustive search with a proof of completeness has been part of pure mathematics since the four-color theorem of 1976 [68, 69]. Besides exhaustive search we use creative telescoping [70, 55], a computer-algebra method for proving that an integral or sum satisfies a given differential equation or recurrence, and dynamic programming [71], which finds exact optima over ordered partitions.

5.1 Complete censuses and planted instances

A finite enumeration (a census) can only be considered rigorously complete if it comes with a proof that it missed nothing. Such a proof is an identity that the full list satisfies and a partial list cannot. For integral lattices the identity is Siegel’s mass formula [72], which fixes in advance a weighted count of all lattice classes of a given local type (a genus). An enumeration is complete exactly when the weights of the classes found add up to the mass. As an additional check, if a second, independent enumeration algorithm exists the two lists must coincide.

A claim that no solution exists needs a proof of a different kind, a theorem rather than a checked list (Section 5.2). Enumerations and nonexistence results are rigorous as mathematics: completeness is proved, not sampled, and nothing can be missed if the program does what the

proof says. However, when AI is doing the proving, there is another failure mode: bugs in the code. A computer-assisted proof is only as reliable as the code that executes it. A program that silently skips part of a search space, not because it didn't know that space was there, but because of a bug in the code, may mistakenly claim it has been exhaustive. Thus it is important to have controls so that this doesn't happen. One powerful way to do this is to plant synthetic targets of the same kind and size and make sure the solver finds them. The planted control tests the implementation, not the argument; this is a standard precaution of computer-assisted proofs. Note that a heuristic search gives no such guarantee even with correct code: a Markov chain Monte Carlo sampler can pass its convergence diagnostics while missing a well-separated mode, which is one reason we enumerate rather than sample where enumeration is possible.

For a concrete example, consider string compactifications with flux. There, the allowed fluxes form a lattice (each point a candidate vacuum), and such vacua are usually explored one at a time [11]. To answer a question about all of them at once (whether any allowed flux stabilizes every modulus, for example) we need the complete list. Here the completeness identity is again Siegel's mass formula, and a BootLoops routine Terrier, named after the string landscape, can carry out the enumeration in exact lattice arithmetic. Another example concerns the mysterious Voynich manuscript. Readings of this manuscript can be written as probability models, fitted on part of the pages and judged on the rest [73]. A failure on the withheld pages excludes a reading only if the test has also been shown to detect disguised real text of the same kind (the planted-instance rule).

An important application is to the operating characteristics of adaptive and Bayesian Phase II trial designs. The type I error rate and power are usually estimated from a few thousand simulated trials, each with Monte Carlo noise [12]. The actual decision rule can often be stated as requiring either finite sums over outcome sequences or low-dimensional integrals. These can then be computed exactly or as proven enclosures.

A final application is to Medicare's star-rating systems, which grade one-dimensional quality scores by cut points, defined as the boundaries of the bins that minimize the within-bin sum of squared deviations. In practice the bins are computed with clustering heuristics that may stop short of that minimum [13]. In one dimension, however, the optimal bins are intervals, and the exact minimum has been computable since 1958 by a dynamic program over the cut points [74]. That dynamic program is an exhaustive search over every placement of the cut points, organized so that it runs in polynomial time, and it returns the global minimum together with the proof that no other placement does better; a clustering heuristic returns a local minimum and no such statement.

5.2 Impossibility as a theorem

Exhaustive enumeration can also help BootLoops compute integrals, particularly by the constraint leg of the bootstrap (Section 3.2). It may happen that in exhausting a class of candidate formulas for a given integral one finds, instead of a list, a proof that no member of the class equals the integral. This happened in the computation of the anisotropic Watson integral,

$$W(a, b, c; w) = \frac{1}{\pi^3} \int_0^\pi \int_0^\pi \int_0^\pi \frac{d\theta_1 d\theta_2 d\theta_3}{w - a \cos \theta_1 - b \cos \theta_2 - c \cos \theta_3}, \quad (12)$$

the generating function of the return probabilities of a random walk on the simple cubic lattice that hops along the three axes with rates (a, b, c) . Whether a closed form exists at generic rates has been an open question since Watson evaluated the isotropic case $a = b = c$ in 1939 [75]. Every previously known exact result lies on a locus where two rates coincide, on which the integral factors into two complete elliptic integrals [76, 77].

The integral of Eq. (12) is holonomic in w (it satisfies a linear differential equation with polynomial coefficients). The kinds of closed form it can have depend on the order and the reducibility of the minimal such equation. From the series coefficients of W we obtained, in exact arithmetic, a candidate annihilating operator of order 5. We then proved by creative telescoping that it annihilates W : applied to the integrand, the operator gives an exact total derivative in the angles [55, 78, 79]. For both steps, and for the proof that no operator of lower order exists, we again used Annihilator (Table 1), the routine written for Feynman integrals and reused above for phylogenetic evidence.

The impossibility argument then runs as follows. A complete elliptic integral satisfies a differential equation of order 2, and any candidate of the sought form (a product of two complete elliptic integrals with an algebraic prefactor) therefore satisfies an equation of order 4. The minimal operator of W has order 5 and is irreducible for general rates: its differential Galois group is the full $SO(5)$, which excludes every function built from solutions of second-order equations. No product of complete elliptic integrals and no ${}_3F_2$ represents the generic Watson integral. At one specially constructed rate tuple with complex multiplication the integral has a closed form in $\Gamma(k/16)$ values, and for general rates the integral is a period of a K3 surface [79].

Thus the nonexistence of the sought closed form is a theorem about the orders of differential operators, and its proof is produced by computer algebra. The telescoping identity on which the proof rests can be confirmed by differentiation alone, without the language model. A class of closed forms can also be excluded by a linear-independence obstruction, when the differential equation requires a function that no combination of members of the class can equal, as for the crossed light-by-light box of Section 3.4, whose top-sector cut is polylogarithmic but which has no polylogarithmic form over its own letters because of a K3 sub-sector (Table 3).

6 Ball arithmetic and audits

Many calculations in quantitative science end in a number computed in floating-point arithmetic, with a rounding error that is neither estimated nor propagated. Rigorous alternatives have existed since the interval analysis of the 1960s [80]. In validated numerics, bounds on rounding error are carried through entire computations and into computer-assisted proofs [81, 82]. The Arb library [32] made interval arithmetic in ball form (a midpoint with a radius, in place of two endpoints) efficient at arbitrary precision. We use ball arithmetic to compute numbers with proven error bounds and to examine numbers already in print.

6.1 Balls and Muller’s recurrence

A ball is a midpoint–radius pair $m \pm r$. It denotes the interval $[m - r, m + r]$, and an arithmetic operation $\circ$ on two balls is defined to return a ball that contains the exact result for every choice of one member from each ball,

$$x \in [m_1 \pm r_1], \ y \in [m_2 \pm r_2] \implies x \circ y \in [m_1 \pm r_1] \circ [m_2 \pm r_2]. \quad (13)$$

Elementary and special functions extend to balls in the same way [80, 32]. Every operation on balls rounds the radius outward, which keeps the true value within r of m through any sequence of operations; $\log_{10}(|m|/r)$ is then the number of guaranteed digits. Our implementation, Baller [34], is built on Arb; given a requested number of digits, it raises the working precision until that many digits are guaranteed, and otherwise returns a refusal naming the test that failed.

Muller's recurrence: $x_{n+1} = 111 - \frac{1130}{x_n} + \frac{3000}{x_n x_{n-1}}$, $x_0 = 2$, $x_1 = -4$				
true answer	$x_{100} =$	$\frac{2939933766307273816384515931624831928277835213467975166492673610148790603445067}{489988959736444127362399646251257712574995486122651802567073927074333549467407} = 6.000000016099565$		
		x_{30}	x_{100}	
fixed precision	16 digits	99.999999999999940	100.000000000000000	leaves 6 at $n = 16$
	50 digits	6.0056486887714203	100.000000000000000	leaves 6 at $n = 45$
	300 digits	6.0056486887714203	6.0000000160995649	leaves 6 at $n = 250$
ball arithmetic	16 digits	99.999999999999940 $\pm \infty$	100.000000000000000 $\pm \infty$	a divisor ball contains 0
	50 digits	6.0056486887714211 $\pm 3 \times 10^{-6}$	100.000000000000000 $\pm \infty$	a divisor ball contains 0
	300 digits	6.0056486887714203 $\pm 2 \times 10^{-256}$	6.0000000160995649 $\pm 3 \times 10^{-140}$	certified

Figure 4: At 16 and 50 digits of working precision, fixed-precision arithmetic returns a wrong value of x_{100} in Muller’s recurrence, Eq. (14), with no indication that it is wrong, whereas ball arithmetic returns either certified digits or an infinite radius. Top: the recurrence with its seeds, and the exact x_{100} , a ratio of a 79-digit integer to a 78-digit integer, approximately 6.000000016099565. Middle: x_{30} and x_{100} computed in fixed precision at 16, 50 and 300 digits. At 16 and 50 digits x_{100} prints as exactly 100, because rounding error excites a second solution of the recurrence that grows about seventeen-fold per step; at 300 digits the value is correct. The notes at right give the step at which each fixed-precision iterate leaves the neighborhood of 6. Bottom: the same three computations in ball arithmetic. Where a divisor ball contains zero the radius is infinite and no digit is asserted; at 300 digits every printed digit of x_{100} is guaranteed. The midpoints in the bottom rows track the fixed-precision values because a ball’s midpoint is computed by the same floating-point operations; only the radius indicates which of its digits are established.

We illustrate the role of the radius with Muller’s recurrence, a standard example in numerical analysis. The sequence

$$x_{n+1} = 111 - \frac{1130}{x_n} + \frac{3000}{x_n x_{n-1}}, \quad x_0 = 2, \quad x_1 = -4, \quad (14)$$

converges to 6, and the true x_{100} is an exact fraction of 79- and 78-digit integers, approximately 6.000000016099565 (Fig. 4). Any rounding error excites a second solution of the recurrence that tends to 100 and grows about seventeen-fold per step relative to the true one. In fixed-precision arithmetic with about a hundred digits or fewer the computed x_{100} is therefore near 100, and only beyond roughly 120 digits does it settle near 6. With 300 digits the computed value is correct, but neither output gives any indication of its accuracy.

By contrast, iterating Eq. (14) in ball arithmetic gives the same midpoints as the fixed-precision computation, each accompanied by a radius. At 16 and 50 digits of working precision a divisor ball eventually contains zero and the radius becomes infinite; the ball still contains the true value but guarantees no digit of it. Asked for 16 digits of x_{100} , Baller raises its working precision from 30 to 240 decimal digits and certifies $x_{100} = 6.0000000160995649$; the same program in ordinary floating point returns 100.

Computing with more digits than seem necessary is the usual precaution, but the result carries no bound on how many of its digits are correct, and a single subtraction of nearly equal quantities can cancel thirty of them. Rerunning at higher precision to see whether the answer changes is impractical for a computation of millions of operations. In ball arithmetic the radius bounds the

true error after each operation, so that the working precision can be raised exactly where a ball becomes too wide and the final enclosure can be used in a proof.

In particular, an interval-Newton step, the Krawczyk test [83], can prove that a given box (a product of intervals) contains exactly one zero of a system of equations. Applied to the stationarity or equilibrium conditions of a model, it yields a proof that an optimum or an equilibrium, first located numerically, exists and is unique within the box. The test requires a preconditioning matrix. A poorly chosen matrix may make it fail, but when it passes, the conclusion is proved whatever matrix was used. We use the test for the equilibria of a minimal model of atmospheric oxygen [84]. The same test applies to the critical point of the superpotential at a candidate string vacuum, whose existence depends on inequalities among periods known only numerically [11]. However, enclosures can establish inequalities only: two disjoint balls prove that two numbers differ, whereas two overlapping balls of positive radius are consistent both with equal and with slightly different values of the two numbers. Equalities in our work come instead from exact rational arithmetic, or from an identity proposed by an integer-relation fit, Eq. (3), and then either proved or accepted on passing the numerical test of Eq. (4).

6.2 Correlated factors and Taylor forms

Naive ball arithmetic can fail by overestimation, returning an enclosure that is correct but far too wide to use. The common case in statistical work is a likelihood formed as a product of hundreds of per-observation factors that all depend on one shared parameter (a rate, say). Interval evaluation treats the factors as unrelated and adds the magnitudes of their first-order variations, which in the true product largely cancel. In one likelihood of this form the summed variations gave an enclosure of width 10^{22} , although the likelihood’s true sensitivity to the parameter was a few hundred.

The remedy for this overestimation is a classical device, the centered Taylor form [85, 81]. It carries the dependence on the shared parameter as a truncated series of stated order with a rigorous bound on the remainder, and the cancellations then occur among series coefficients before any interval is widened. We packaged this construction as a routine, ERAS (for exact remainder-aware series), together with a set of test problems with known answers that any adaptation of it must reproduce. The routine was written when the failure first appeared, in a likelihood computation on linguistic data. It has since been reused for the arithmetic of the statistical applications of Section 4, among them the mixture-model and population-genetics computations [9, 67].

6.3 Audits of published numbers

A second use of exact and ball arithmetic is the audit of numbers already published (regulatory thresholds, quality ratings, reference tables). Such an audit again yields either proven digits or a refusal, and we report its result as one of four classes: the number in question can be reproduced from the published inputs; it is determined only under unstated conventions; it is undetermined by what was published; or it cannot be examined at all. When our recomputation disagrees with a published value, we state the criterion applied, the step of the calculation at which the discrepancy arises, and its size, in enough detail for a reader to reproduce the discrepancy from the same inputs. A routine written for the purpose, Recount, replays published arithmetic in exact rational numbers and thereby excludes our own rounding as a source of discrepancy.

Two examples from public health concern published numbers defined by a stated rule. The cut points of Medicare’s star-rating systems are defined by the clustering criterion of Section 5.1, and whether a published cut point coincides with the optimum of that criterion can be decided exactly [13]. United States law limits ozone in outdoor air to 70 parts per billion. Whether a

monitoring site attains the standard is judged by a three-year statistic of its readings (the design value), which under the regulation is truncated at its last digit after digits have already been discarded at earlier steps of the calculation. For a site near the limit, attainment can turn on the final digit alone. To determine how truncation rather than rounding affects attainment, we replay the whole chain of calculations in exact arithmetic from the public monitoring data [14].

For another example, consider population genetics. There, the standard programs that infer the distribution of fitness effects from observed site-frequency spectra, among them polyDFE [86], dadi [87] and fastDFE [88], work in floating-point arithmetic. The exact spectra of Section 4.3 supply reference values against which the range of validity of such a program may be mapped [67]. The operating characteristics of a Bayesian clinical-trial design depend on Beta and Dirichlet tail integrals, which can likewise be computed with proven error bounds; Ref. [12] shows what exact arithmetic adds to the analysis of such trials.

7 Conclusions

The thesis of this paper is that there is tremendous and largely underutilized potential in the breadth of knowledge of agentic AI. When this knowledge is mixed with the ability of the current generation of AI tools to write computer code and do technically challenging computations in mathematical physics, the combination is strikingly powerful. This viewpoint does not attempt to push out the deepest boundaries of human knowledge in one direction, but rather combines knowledge that already exists in different fields. That is, it allows us to fill in the convex hull of humanity’s scientific corpus. This hull is enormous and much of it is unvisited: it contains every combination of a method from one field with a question from another, and in many pairings there is hardly anyone who knows both. The reduction algorithms of collider physics and the evidence integrals of phylogenetics are one example, interval arithmetic and population genetics another. A model that has read everything can bring everything together.

BootLoops is a first attempt at bringing together tools from distant areas to make it easier for scientists to make progress on long-standing problems, or to find new ones. BootLoops is meant to grow, as more methods are found and incorporated. Or other similar harnesses may be built elsewhere with other tools for other things. The hull is convex, but the possibilities are unbounded.

BootLoops is discussed in more detail at <https://bootloops.ai>. The code and forks of public programs are at <https://github.com/BootLoops-ai>.

Acknowledgments

We would like to thank the following people for advice and collaboration: Isaiah Andrews, Nima Arkani-Hamed, Michael Desai, Ethan Dyer, Scott Edwards, Noam Elkies, Cecilia Garraffo, Matthew Gordon, Thomas Grimm, Martin Hemberg, Mikhail Ivanov, David Johnston, Gary King, Paul Lewis, Amara McCune, Brendan Meade, Siddharth Mishra-Sharma, James O’Dwyer, Joe Pater, Kevin Ryan, Subhabrata Sen, Jesse Shapiro, and Xiaoyuan Zhang.

This paper was written with the assistance of generative AI, Claude Fable 5.1 (Anthropic). The BootLoops package was written by AI, except for forks of public packages which are their authors’ work. The author takes full responsibility for the contents of this paper and of BootLoops. Funding for this project was provided by Anthropic. This is not an official Anthropic project and results and views are not endorsed by Anthropic.

References

- [1] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.
- [2] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. Learning skillful medium-range global weather forecasting. *Science*, 382:1416–1421, 2023.
- [3] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gwooon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624:80–85, 2023.
- [4] Samuel Abreu, Ruth Britto, and Claude Duhr. The SAGEX review on scattering amplitudes, Chapter 3: Mathematical structures in Feynman integrals. 2022. arXiv:2203.13014, J. Phys. A **55** (2022) 443004.
- [5] Lance J. Dixon, James M. Drummond, and Johannes M. Henn. Bootstrapping the three-loop hexagon. 2011. arXiv:1108.4461, JHEP **11** (2011) 023.
- [6] Holmfridur S. Hannesdottir, Andrew J. McLeod, Matthew D. Schwartz, and Cristian Vergu. Applications of the Landau bootstrap. 2024. arXiv:2410.02424, Phys. Rev. D **111** (2025) 085003.
- [7] Oscar Barrera, Aurélien Dersy, Rabia Husain, Matthew D. Schwartz, and Xiaoyuan Zhang. Analytic regression of Feynman integrals from high-precision numerical sampling. 2025. arXiv:2507.17815, JHEP **01** (2026) 014.
- [8] Matthew D. Schwartz, Scott V. Edwards, and Paul O. Lewis. Exact Bayesian evidence for phylogenetic trees. Companion paper, <https://bootloops.ai/exactphylogenetics.pdf>, 2026.
- [9] Matthew D. Schwartz and Subhabrata Sen. Exact Bayesian evidence for mixture models: from two components to infinity. Companion paper, <https://bootloops.ai/mixtures.pdf>, 2026.
- [10] Matthew D. Schwartz and Martin Hemberg. Controlled inference of transcriptional bursting from single-cell RNA counts. Companion paper, <https://bootloops.ai/genebursting.pdf>, 2026.
- [11] Matthew D. Schwartz. Exact methods for string flux vacua. Companion paper, <https://bootloops.ai/stringvac.pdf>, 2026.
- [12] Matthew D. Schwartz. Exact versus simulated operating characteristics of published adaptive and Bayesian trial designs. Companion paper, <https://bootloops.ai/healthtrials.pdf>, 2026.
- [13] Matthew D. Schwartz. Stars misaligned: Medicare star ratings and the exact optimum of their clustering criterion. Companion paper, <https://bootloops.ai/healthstars.pdf>, 2026.
- [14] Matthew D. Schwartz. How truncation and rounding affect attainment of the U.S. ozone standard. Companion paper, <https://bootloops.ai/airquality.pdf>, 2026.

- [15] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [16] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. 2024. arXiv:2408.06292 [cs.AI].
- [17] Ali E. Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Dmytro Shved, Gavin J. Gyimesi, Jon M. Laurent, Samantha M. Wright, Muhammed T. Razzak, Andrew D. White, Silvia C. Finnemann, Michaela M. Hinks, and Samuel G. Rodrigues. A multi-agent system for automating scientific discovery. *Nature*, 655:497–505, 2026.
- [18] Anthropic. Claude Science, an AI workbench for scientists. <https://www.anthropic.com/news/claude-science-ai-workbench>, 2026.
- [19] OpenAI. Introducing Codex. <https://openai.com/index/introducing-codex/>, 2025.
- [20] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite-field methods. 2021. arXiv:2008.06494, Comput. Phys. Commun. **266** (2021) 108024.
- [21] Claudia Fevola, Sebastian Mizera, and Simon Telen. Principal Landau determinants. 2023. arXiv:2311.16219, Comput. Phys. Commun. **303** (2024) 109278; see also 2311.14669.
- [22] M. Correia, M. Giroux, and S. Mizera. SOFIA: Singularities of Feynman integrals automatized. 2025. arXiv:2503.16601, Comput. Phys. Commun. **320** (2026) 109970.
- [23] Xin Guan, Xiao Liu, Yan-Qing Ma, and Wen-Hao Wu. Blade: A package for block-triangular form improved Feynman integrals decomposition. 2024. arXiv:2405.14621, Comput. Phys. Commun. **310** (2025) 109538.
- [24] Xiao Liu and Yan-Qing Ma. AMFlow: a Mathematica package for Feynman integrals computation via auxiliary mass flow. 2022. arXiv:2201.11669, Comput. Phys. Commun. **283** (2023) 108565.
- [25] AMFlow.cpp contributors. AMFlow.cpp: a C++ reimplementation of AMFlow, version 1.1.0. <https://github.com/chang18/amflow-cpp>, 2026.
- [26] Jonas Klappert and Fabian Lange. Reconstructing rational functions with FireFly. 2019. arXiv:1904.00009, Comput. Phys. Commun. **247** (2020) 106951.
- [27] Matthew D. Schwartz. Seedling: learning how much to stage in integration-by-parts reduction. Companion paper, <https://bootloops.ai/seedling.pdf>, 2026.
- [28] S. Borowka, G. Heinrich, S. Jahn, S. P. Jones, M. Kerner, J. Schlenk, and T. Zirke. pySecDec: a toolbox for the numerical evaluation of multi-scale integrals. 2017. arXiv:1703.09692, Comput. Phys. Commun. **222** (2018) 313–326.
- [29] A. V. Smirnov, N. D. Shapurov, and L. I. Vysotsky. FIESTA5: numerical high-performance Feynman integral evaluation. 2021. arXiv:2110.11660, Comput. Phys. Commun. **277** (2022) 108386.

- [30] H. R. P. Ferguson, D. H. Bailey, and S. Arno. Analysis of PSLQ, an integer relation finding algorithm. *Math. Comp.*, 68:351–369, 1999.
- [31] A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovász. Factoring polynomials with rational coefficients. *Math. Ann.*, 261:515–534, 1982.
- [32] Fredrik Johansson. Arb: efficient arbitrary-precision midpoint-radius interval arithmetic. *IEEE Transactions on Computers*, 66:1281–1292, 2017.
- [33] The mpmath development team. *mpmath: a Python library for arbitrary-precision floating-point arithmetic (version 1.4.0)*, 2026. <http://mpmath.org/>.
- [34] Matthew D. Schwartz. Baller: fail-closed ball arithmetic for scientific computing. Companion paper, <https://bootloops.ai/baller.pdf>, 2026.
- [35] Matthew D. Schwartz, Scott V. Edwards, and Paul O. Lewis. JaCK & Jill: exact marginal likelihoods for small phylogenetic trees. Companion paper, <https://bootloops.ai/jackandjill.pdf>, 2026.
- [36] Nicolas Lartillot and Hervé Philippe. Computing Bayes factors using thermodynamic integration. *Syst. Biol.*, 55:195–207, 2006.
- [37] Wangang Xie, Paul O. Lewis, Yu Fan, Lynn Kuo, and Ming-Hui Chen. Improving marginal likelihood estimation for Bayesian phylogenetic model selection. *Syst. Biol.*, 60:150–160, 2011.
- [38] Mathieu Fourment, Andrew F. Magee, Chris Whidden, Arman Bilge, Frederick A. Matsen, IV, and Vladimir N. Minin. 19 dubious ways to compute the marginal likelihood of a phylogenetic tree topology. *Syst. Biol.*, 69:209–220, 2020.
- [39] Martin Eichler. Eine Verallgemeinerung der Abelschen Integrale. 1957. *Math. Z.* **67** (1957) 267–298.
- [40] Shaowei Lin, Bernd Sturmfels, and Zhiqiang Xu. Marginal likelihood integrals for mixtures of independence models. *J. Mach. Learn. Res.*, 10:1611–1631, 2009.
- [41] R. J. Eden, P. V. Landshoff, D. I. Olive, and J. C. Polkinghorne. *The Analytic S-Matrix*. Cambridge University Press, 1966.
- [42] I. M. Gel’fand, M. M. Kapranov, and A. V. Zelevinsky. Generalized Euler integrals and A-hypergeometric functions. *Adv. Math.*, 84:255–271, 1990.
- [43] Roman N. Lee and Andrei A. Pomeransky. Critical points and number of master integrals. 2013. arXiv:1308.6676, JHEP **11** (2013) 165.
- [44] Sebastian Mizera and Simon Telen. Landau discriminants. 2021. arXiv:2109.08036, JHEP **08** (2022) 200.
- [45] S. Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. 2000. arXiv:hep-ph/0102033, Int. J. Mod. Phys. A **15** (2000) 5087–5159.
- [46] Alexander B. Goncharov, Marcus Spradlin, C. Vergu, and Anastasia Volovich. Classical polylogarithms for amplitudes and Wilson loops. 2010. arXiv:1006.5703, Phys. Rev. Lett. **105** (2010) 151605.

- [47] Johannes M. Henn. Multiloop integrals in dimensional regularization made simple. 2013. arXiv:1304.1806, Phys. Rev. Lett. **110** (2013) 251601.
- [48] Luise Adams and Stefan Weinzierl. Feynman integrals and iterated integrals of modular forms. 2017. arXiv:1704.08895, Commun. Number Theory Phys. **12** (2018) 193–251.
- [49] Spencer Bloch and Pierre Vanhove. The elliptic dilogarithm for the sunset graph. 2015. arXiv:1309.5865, J. Number Theory **148** (2015) 328–364.
- [50] Philip Candelas, Xenia C. de la Ossa, Paul S. Green, and Linda Parkes. A pair of Calabi–Yau manifolds as an exactly soluble superconformal theory. 1991. Nucl. Phys. B **359** (1991) 21–74.
- [51] Kilian Bönisch, Fabian Fischbach, Albrecht Klemm, Christoph Nega, and Reza Safari. Analytic structure of all loop banana integrals. 2020. arXiv:2008.10574, JHEP **05** (2021) 066.
- [52] Maxim Kontsevich and Don Zagier. Periods. In *Mathematics Unlimited — 2001 and Beyond*, pages 771–808. Springer, 2001.
- [53] Roman N. Lee. Reducing differential equations for multiloop master integrals. *JHEP*, 04:108, 2015. arXiv:1411.0911.
- [54] T. Peraro. Scattering amplitudes over finite fields and multivariate functional reconstruction. *JHEP*, 12:030, 2016.
- [55] C. Koutschan. Creative telescoping for holonomic functions. In C. Schneider and J. Blümlein, editors, *Computer Algebra in Quantum Field Theory: Integration, Summation and Special Functions*, pages 171–194. Springer Wien, 2013.
- [56] S. Laporta. High-precision calculation of the 4-loop contribution to the electron $g - 2$ in QED. 2017. arXiv:1704.06996, Phys. Lett. B **772** (2017) 232–238.
- [57] Matthew D. Schwartz. Polylogarithmic, elliptic, and Calabi–Yau Feynman integrals from a hybrid bootstrap. Companion paper, <https://bootloops.ai/SmatrixBootLoops.pdf>, 2026.
- [58] Matthew D. Schwartz. Automated computation of Feynman integrals with BootLoops. Companion paper, <https://bootloops.ai/FeynmanBootstrap.pdf>, 2026.
- [59] Matthew D. Schwartz. Bootstrapping elliptic and Calabi–Yau Feynman integrals. Companion paper, <https://bootloops.ai/FeynmanGeometry.pdf>, 2026.
- [60] Robert E. Kass and Adrian E. Raftery. Bayes factors. *J. Amer. Statist. Assoc.*, 90:773–795, 1995.
- [61] Bernd Sturmfels and Seth Sullivant. Toric ideals of phylogenetic invariants. *J. Comput. Biol.*, 12:204–228, 2005.
- [62] Fredrik Ronquist, Maxim Teslenko, Paul van der Mark, Daniel L. Ayres, Aaron Darling, Sebastian Höhna, Bret Larget, Liang Liu, Marc A. Suchard, and John P. Huelsenbeck. MrBayes 3.2: efficient Bayesian phylogenetic inference and model choice across a large model space. *Syst. Biol.*, 61:539–542, 2012.
- [63] Matthew D. Schwartz. Measuring shared authorship in the Federalist Papers, Shakespeare and the Bible. Companion paper, <https://bootloops.ai/authors.pdf>, 2026.

- [64] Stephen P. Hubbell. *The Unified Neutral Theory of Biodiversity and Biogeography*, volume 32 of *Monographs in Population Biology*. Princeton University Press, 2001.
- [65] Rampal S. Etienne. A new sampling formula for neutral biodiversity. *Ecology Letters*, 8:253–260, 2005.
- [66] Anton J. M. Larsson, Per Johnsson, Michael Hagemann-Jensen, Leonard Hartmanis, Omid R. Faridani, Björn Reinius, Åsa Segerstolpe, Chloe M. Rivera, Bing Ren, and Rickard Sandberg. Genomic encoding of transcriptional burst kinetics. *Nature*, 565:251–254, 2019.
- [67] Matthew D. Schwartz and Michael M. Desai. An exact solution for the site-frequency spectrum of a selected allele. Companion paper, <https://bootloops.ai/mutationflow.pdf>, 2026.
- [68] K. Appel and W. Haken. Every planar map is four colorable. *Bull. Amer. Math. Soc.*, 82:711–712, 1976.
- [69] K. Appel and W. Haken. Every planar map is four colorable. Part I: Discharging. *Illinois J. Math.*, 21:429–490, 1977.
- [70] M. Petkovšek, H. S. Wilf, and D. Zeilberger. *A = B*. A K Peters/CRC Press, 1996. Full text free at <https://www2.math.upenn.edu/~wilf/AeqB.html>.
- [71] R. E. Bellman. *Dynamic Programming*. Princeton University Press, 1957. Reprinted in Princeton Landmarks in Mathematics and Physics, 2010.
- [72] Carl Ludwig Siegel. Über die analytische Theorie der quadratischen Formen. *Ann. of Math.*, 36:527–606, 1935.
- [73] Matthew D. Schwartz. How the Voynich Manuscript was written. Companion paper, <https://bootloops.ai/voynich.pdf>, 2026.
- [74] Walter D. Fisher. On grouping for maximum homogeneity. *J. Amer. Statist. Assoc.*, 53:789–798, 1958.
- [75] G. N. Watson. Three triple integrals. *Q. J. Math. (Oxford)*, os-10:266–276, 1939.
- [76] R. T. Delves and G. S. Joyce. On the Green function for the anisotropic simple cubic lattice. *Ann. Phys.*, 291:71–133, 2001.
- [77] I. J. Zucker. 70+ years of the Watson integrals. *J. Stat. Phys.*, 145:591–612, 2011.
- [78] M. Kauers and P. Paule. *The Concrete Tetrahedron: Symbolic Sums, Recurrence Equations, Generating Functions, Asymptotic Estimates*. Texts and Monographs in Symbolic Computation. Springer Vienna, 2011.
- [79] Noam Elkies, Thomas W. Grimm, and Matthew D. Schwartz. The anisotropic Watson integral. In preparation, 2026.
- [80] R. E. Moore, R. B. Kearfott, and M. J. Cloud. *Introduction to Interval Analysis*. SIAM, 2009.
- [81] W. Tucker. *Validated Numerics: A Short Introduction to Rigorous Computations*. Princeton University Press, 2011.
- [82] S. M. Rump. Verification methods: rigorous results using floating-point arithmetic. *Acta Numerica*, 19:287–449, 2010.

- [83] R. Krawczyk. Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken. *Computing*, 4:187–201, 1969.
- [84] Matthew D. Schwartz and David T. Johnston. The Great Oxidation from a minimal ocean–atmosphere model. Companion paper, <https://bootloops.ai/greatoxidation.pdf>, 2026.
- [85] Martin Berz and Kyoko Makino. Verified integration of ODEs and flows using differential algebraic methods on high-order Taylor models. *Reliable Computing*, 4:361–369, 1998.
- [86] Paula Tataru, Maéva Mollion, Sylvain Glémin, and Thomas Bataillon. Inference of distribution of fitness effects and proportion of adaptive substitutions from polymorphism data. *Genetics*, 207:1103–1119, 2017.
- [87] Ryan N. Gutenkunst, Ryan D. Hernandez, Scott H. Williamson, and Carlos D. Bustamante. Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genet.*, 5:e1000695, 2009.
- [88] Janek Sendrowski and Thomas Bataillon. fastDFE: fast and flexible inference of the distribution of fitness effects. *Mol. Biol. Evol.*, 41(5):msae070, 2024.